

1. Rushikesh — Order Service + Business Workflow

Ownership

You should own the central business workflow.

Service

order-service

Database

order_db

|—— orders

|—— order_items

|—— order_status_history

Main APIs

POST /api/orders

GET /api/orders/{id}

GET /api/orders/user/{userId}

PUT /api/orders/{id}/cancel

GET /api/orders/{id}/status

Your main responsibility

You coordinate the business flow:

Customer



Create Order



Validate Product



Reserve Inventory



Process Payment



Confirm Order



Send Notification

Kafka Events

Produce:

OrderCreated

OrderCancelled

OrderConfirmed

Consume:

InventoryReserved

InventoryFailed

PaymentSuccessful

PaymentFailed

Why you own this

Order Service connects almost every business component, so you should define the overall business flow and API/event contracts.

2. Rohit — User + Authentication + Security

Ownership

user-service

security

JWT

Database

user_db

|—— users

└── addresses

APIs

POST /api/auth/register

POST /api/auth/login

GET /api/users/{id}

PUT /api/users/{id}

POST /api/users/{id}/addresses

GET /api/users/{id}/addresses

PUT /api/users/{id}/addresses/{addressId}

DELETE /api/users/{id}/addresses/{addressId}

Security

Spring Security

↓

Authentication

↓

JWT

↓

Authorization

↓

RBAC

Roles:

CUSTOMER

ADMIN

Day-5 expectation

Other services should be able to receive:

Authorization: Bearer <JWT>

and protected APIs should work.

3. Raj — Product & Catalog

Ownership

product-service

Database

product_db

|—— categories

|—— products

|—— product_images

APIs

POST /api/categories

GET /api/categories

GET /api/categories/{id}

POST /api/products

GET /api/products

GET /api/products/{id}

PUT /api/products/{id}

DELETE /api/products/{id}

Important API for Order Service

GET /api/products/{productId}

Response:

```
{  
  "id": "uuid",  
  "name": "Laptop",
```

```
"sku": "LAP-001",  
"price": 75000,  
"currency": "INR",  
"status": "ACTIVE"  
}
```

Additional responsibility

Implement:

Pagination

Sorting

Searching

Redis caching

Example:

GET /api/products?page=0&size=10

4. Pranav — Inventory & Stock

Ownership

inventory-service

Database

inventory_db

|—— inventory

|—— inventory_reservations

APIs

GET /api/inventory/{productId}

POST /api/inventory/reserve

POST /api/inventory/release

POST /api/inventory/confirm

Business logic

Suppose:

Available = 10

Customer orders 2:

Available = 8

Reserved = 2

Payment succeeds:

Reserved → Confirmed

Payment fails:

Reserved → Released

Available = 10

Important responsibility

Handle concurrent orders.

Example:

Stock = 5

Customer A → buys 4

Customer B → buys 4

You must prevent:

Stock = -3 ❌

Use optimistic locking:

@Version

private Long version;

 5. Mayur — Payment + Notification

Mayur owns two closely related services.

Payment Service

Database

payment_db

└── payments

└── payment_transactions

APIs

POST /api/payments

GET /api/payments/{id}

POST /api/payments/{id}/refund

Status

INITIATED

PROCESSING

SUCCESS

FAILED

REFUNDED

For a 5-day project, use a mock payment gateway.

Order Service

↓

Payment Service

↓

Mock Gateway

↓

SUCCESS / FAILED

Notification Service

Database

notification_db

└── notifications

Consumes Kafka events:

OrderCreated

OrderConfirmed

OrderCancelled

PaymentSuccessful

PaymentFailed

Then:

Kafka

↓

Notification Service

↓

Email / SMS / Push

Initially you can implement email as a mock/logging implementation.

DAY-BY-DAY PLAN

The most important thing is that every day has a business goal.

DAY 1 — Project Foundation + Individual Services

Rushikesh

Create Order Service

Create:

orders

order_items

order_status_history

Implement:

Create Order

Get Order

Get User Orders

Cancel Order

Rohit

Create User Service

Create:

users

addresses

Implement:

Register

Login

User CRUD

Address CRUD

Start JWT implementation

Raj

Create Product Service

Create:

categories

products

product_images

Implement:

Category CRUD

Product CRUD

Pranav

Create Inventory Service

Create:

inventory

inventory_reservations

Implement:

Stock CRUD

Get Stock

Mayur

Create Payment Service

Create:

payments

payment_transactions

Create Notification Service

Create:

notifications

Shared infrastructure

One developer should create the initial skeleton for:

Eureka Server

API Gateway

Config Server

Docker Compose

PostgreSQL

Then the team can integrate it.

 Day 1 result

All services should:

Start

↓

Connect DB

↓

Create Tables

↓

Basic API Works

DAY 2 — REST + Feign Communication

Now we connect the services.

Rushikesh

Implement:

Order

↓

Product Service

and:

Order



Inventory Service

and:

Order



Payment Service

Use OpenFeign.

Rohit

Complete:

JWT

Spring Security

Gateway Authentication

RBAC

Raj

Provide stable Product APIs:

Get Product

Get Product Price

Validate Product

Search Product

Pranav

Implement:

Reserve Stock

Release Stock

Confirm Stock

Insufficient Stock

Mayur

Implement:

Payment Initiation

Payment Success

Payment Failure

Refund

and basic notification processing.

 Day 2 result

You should be able to do:

Client

↓

Gateway

↓

Order

↓

Product

↓

Inventory

↓

Payment

DAY 3 — KAFKA + EVENT-DRIVEN FLOW

This is where the application becomes a proper microservices project.

Kafka topics

order-events

payment-events

inventory-events

notification-events

Main flow

Order Service

|



OrderCreated

|



Kafka

|



Inventory Service

|



InventoryReserved

|



Kafka

|



Payment Service

|

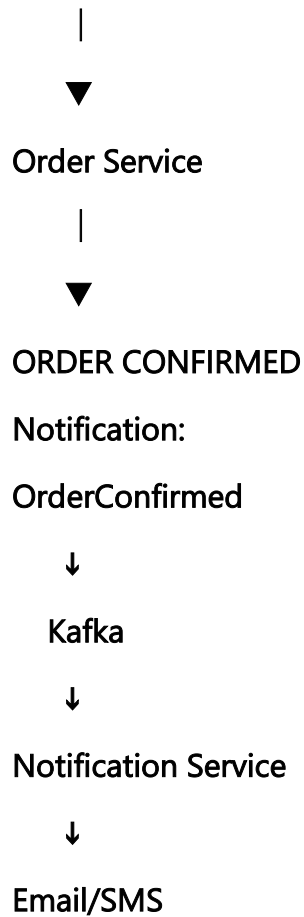


PaymentSuccessful

|



Kafka



● DAY 4 — Failure Handling + Advanced Business Logic

This is the day that makes your project interview-worthy.

Rushikesh

Implement:

Saga workflow

Order cancellation

Order status history

Payment failure handling

Rohit

Implement:

401 Unauthorized

403 Forbidden

JWT expiration

Role-based APIs

Raj

Implement:

Redis

Product caching

Pagination

Sorting

Search

Pranav

Implement:

Optimistic locking

Concurrent stock protection

Inventory rollback

Reservation timeout

Mayur

Implement:

Payment retry

Payment failure

Refund

Notification retry

Kafka consumer error handling

DAY 5 — FULL INTEGRATION + TESTING

No major new features on Day 5.

Everyone focuses on integration.

Final business scenario

Register



Login



JWT



Browse Products



Select Product



Create Order



Reserve Inventory



Make Payment



Payment Success



Confirm Order



Notification

✖ Failure Scenario

You also need to demonstrate:

Create Order



Reserve Inventory



Payment FAILED



Kafka



Release Inventory



Cancel Order



Notification

This demonstrates Saga / distributed transaction handling.

Testing Responsibility

Developer Main Tests

Rushikesh Order + Saga + integration

Rohit JWT + Security + User

Raj Product + Redis + pagination

Pranav Inventory + concurrency

Mayur Payment + Kafka + Notification

Everyone should write:

Unit Tests

Integration Tests

Negative Tests

Team Work Distribution

Rushikesh will be responsible for the **Order Service and overall business workflow**. He will manage order creation, order items, order status, order cancellation, and order history. He will also coordinate the complete business flow between Product, Inventory, and Payment services. His responsibility includes implementing Kafka events such as OrderCreated, OrderConfirmed, and OrderCancelled, and consuming events such as PaymentSuccessful, PaymentFailed, and InventoryReserved.

Rohit will be responsible for the **User Service and Security**. He will develop user registration, login, profile, and address management. He will create the User database containing users and addresses. He will also implement Spring Security, JWT authentication, authorization, and role-based access control for roles such as CUSTOMER and ADMIN. He will work with the API Gateway to make sure only authenticated users can access protected APIs.

Raj will be responsible for the **Product Service**. He will manage categories, products, product images, product prices, SKU, product status, and product search. He will create the Product database and provide APIs that can be consumed by the Order Service. He will also implement pagination, sorting, searching, and Redis caching for frequently accessed product information.

Pranav will be responsible for the **Inventory Service**. He will manage product stock, available quantity, reserved quantity, and sold quantity. His main responsibility is to implement stock reservation, stock release, and stock confirmation. When an order is created, Pranav's service will reserve the required quantity. If payment succeeds, the reservation will be confirmed; if payment fails, the reserved stock will be released. He will also handle concurrent orders using optimistic locking so that stock does not become negative.

Mayur will be responsible for the **Payment Service and Notification Service**. In Payment Service, he will manage payment creation, payment status, transaction details, payment failure, and refund functionality. Initially, a mock payment gateway can be used instead of a real payment provider. In Notification Service, he will consume Kafka events and send notifications when an order is created, payment succeeds or fails, or an order is confirmed or cancelled. He can initially implement email/SMS notification as a mock implementation.

Day 1 — Individual Service Development

On **Day 1**, all developers should create their respective Spring Boot microservices and database structures. Rushikesh will create the Order Service with orders, order_items, and order_status_history tables and implement basic order APIs. Rohit will create the User Service with users and addresses tables and implement registration, login, and basic user APIs. Raj will create the Product Service with categories, products, and product images and implement product CRUD APIs. Pranav will create the Inventory Service with inventory and reservation tables and implement basic inventory APIs. Mayur will create the Payment and Notification services along with their databases. At the same time, the team should set up common infrastructure such as Eureka, API Gateway, PostgreSQL, Kafka, Redis, and Docker Compose.

Day 1 goal: Every service should start successfully, connect to its database, create its tables, and provide basic APIs.

Day 2 — Service-to-Service Communication

On **Day 2**, the team should connect the individual services. Rushikesh will integrate Order Service with Product, Inventory, and Payment using OpenFeign or REST APIs. Rohit will complete JWT authentication, Spring Security, RBAC, and Gateway authentication. Raj will provide stable Product APIs so that Order Service can retrieve product information and prices. Pranav will implement reserve, release, and confirm inventory functionality. Mayur will implement payment initiation, success/failure processing, and basic notification processing.

The main objective is to make the services communicate with each other.

Customer



API Gateway



Order Service



Product Service



Inventory Service



Payment Service

Day 2 goal: The basic synchronous business flow should work from one service to another.

Day 3 — Kafka and Event-Driven Communication

On **Day 3**, the team should introduce Kafka for asynchronous communication. Rushikesh will publish OrderCreated when an order is created. Pranav will consume this event and reserve inventory. After successful reservation, Inventory Service will publish InventoryReserved. Mayur's Payment Service will consume the event and process the payment. After successful payment, Payment Service will publish PaymentSuccessful. Rushikesh will consume this event and confirm the order. Notification Service will consume important events and send notifications.

The main business flow will become:

Order Created



OrderCreated



Inventory Reserved

↓
InventoryReserved
↓
Payment Processed
↓
PaymentSuccessful
↓
Order Confirmed
↓
Notification Sent

Day 3 goal: The complete happy-path order flow should work using Kafka events.

Day 4 — Failure Handling and Production Features

On **Day 4**, the team should focus on real-world scenarios instead of only the successful flow. Rushikesh will implement Saga-style order coordination, cancellation, order status history, and payment-failure handling. Rohit will complete security scenarios such as invalid JWT, expired JWT, 401 Unauthorized, 403 Forbidden, and role-based authorization. Raj will implement Redis caching, pagination, sorting, and product searching. Pranav will handle concurrent stock updates, optimistic locking, reservation release, and inventory rollback. Mayur will implement payment retry, payment failure, refund handling, and notification retry.

For example, if payment fails:

Order Created
↓
Inventory Reserved
↓
Payment Failed
↓
PaymentFailed Event
↓
Inventory Released
↓
Order Cancelled
↓

Notification Sent

Day 4 goal: The application should handle both success and failure scenarios properly.

Day 5 — Complete Integration and Testing

On **Day 5**, no major new functionality should be added. All developers should focus on integrating and testing the complete application. Rohit should test registration, login, JWT, and authorization. Raj should test Product APIs and Redis caching. Pranav should test inventory reservation, release, confirmation, and concurrent orders. Mayur should test payment success, failure, refund, Kafka events, and notifications. Rushikesh should perform end-to-end testing of the complete order workflow.

The final test should look like:

Register



Login



Get JWT



Browse Products



Select Product



Create Order



Reserve Inventory



Make Payment



Payment Success



Confirm Order



Send Notification

After that, the team should run everything through **Docker Compose**, verify Kafka communication, database connections, Redis, Eureka, API Gateway, and all six business services.

Final Responsibility in One Paragraph

In simple business terms, **Rohit handles the customer and authentication, Raj handles the products that the customer wants to buy, Pranav handles whether those products are available in stock, Rushikesh handles what the customer orders and coordinates the complete order workflow, and Mayur handles the customer's payment and notifications.** This distribution gives every developer clear ownership while still allowing the five developers to work together toward one complete application.